TABLE OF CONTENTS

ABSTRACT

In this project, an automatic road quality detection system has been developed using the ESP8266 module, MPU6050 sensor, Ultrasonic sensor, TCRT5000 IR sensor, Active Buzzer, and NEO 6M GPS module. The system aims to detect anomalies and bumps on the road using the accelerometer data from MPU6050, and differentiate between speed breakers and bumps using the ultrasonic sensor. The TCRT5000 IR sensor is used to detect sudden motion and acceleration spikes to identify passing vehicles or pedestrians. The GPS module is used to determine the location of the vehicle, which allows for mapping of the road quality data. The system uses the ESP8266 Wi-Fi module to connect to the internet and upload the data to a Firebase real-time database for analysis and visualization.

The hardware components are connected and assembled on a breadboard, and the software is developed using the Arduino IDE and libraries. The system has been tested in real-time on roads, and the data collected from the sensors has been analyzed and visualized using Firebase. The results of the project show that the system is capable of detecting road anomalies and bumps with high accuracy and provides a comprehensive map of the road quality. This system has the potential to be deployed in real-world scenarios for monitoring road quality and improving road safety.

CHAPTER 1

INTRODUCTION

GENERAL

Driving on smooth roads is something everyone enjoys and certainly provides for a much safer environment. Rough and jagged roads can be hazardous to those who drive on them, leading to the possibility of more accidents, and most significantly, more deaths.

The condition of roadways can vary greatly within both urban and rural cities, and therefore, devices that can aid in quickly identifying hazardous roadways can be extremely valuable to infrastructure engineers striving to increase the safety and quality of their cities. While this prototype device is not fully integrated with urban planners and construction companies, it represents a possible solution to solving one of the primary problems of cities across the world.

Road quality is an essential factor that affects the safety and comfort of drivers and passengers. Therefore, detecting road quality anomalies, such as potholes, speed breakers, and bumps, is crucial for road maintenance and safety. In recent years, advances in sensor technology, wireless communication, and data analysis have made it possible to develop automatic road quality detectors that can monitor the condition of the road surface and report any anomalies in real-time.

In this project, we aim to develop an automatic road quality detector using the ESP8266 microcontroller and various sensors, including the MPU6050 accelerometer, ultrasonic sensor, TCRT5000 IR sensor, and Neo 6M GPS module. The detector will measure the acceleration, distance, and speed of the vehicle to detect bumps,

potholes, speed breakers, and other anomalies in the road surface. It will also use GPS to track the vehicle's location and speed, and IR sensors to detect sudden motions and acceleration spikes.

The system will be powered by the ESP8266 microcontroller, which will collect and process data from the sensors and control the buzzer and LEDs. The data collected will be stored in the SPIFFS file system and uploaded to the Firebase database when the device is connected to Wi-Fi.

PROBLEM DESCRIPTION

Roads are a vital part of modern transportation, and the quality of the roads directly affects the safety and efficiency of the transportation system. Poorly maintained roads can lead to accidents, damage to vehicles, and increased travel time, which can ultimately affect the economy.

However, it can be challenging to monitor the road quality continuously and efficiently. Traditional methods of road quality monitoring involve manual inspections, which can be time-consuming, expensive, and may not provide real-time data.

The aim of this project is to address the challenges associated with traditional road quality monitoring methods by developing an automatic road quality detection system that uses various sensors and modules to collect real-time data on road conditions. The system will provide insights into road conditions, including anomalies, bumps, and speed breakers, and transmit the data to a centralized database in real-time. This data can be used to identify potential hazards on the road and make informed decisions on road maintenance and improvement.

OBJECTIVE

The main objectives of this project are:

1. To design and develop an automatic road quality detection system using various sensors and modules.
2. To detect anomalies and bumps on the road using an accelerometer and differentiate between speed breakers and bumps using an ultrasonic sensor.
3. To use an IR sensor to detect sudden motion and acceleration spikes to identify passing vehicles or pedestrians.
4. To use a GPS module to determine the location of the vehicle and allow for mapping of the road quality data.
5. To use Wi-Fi connectivity to upload the data to a Firebase real-time database for analysis and visualization.
6. To develop a user-friendly interface for accessing and visualizing the road quality data.
7. To provide a comprehensive map of road quality and identify potential hazards on the road.
8. To improve road safety by providing actionable insights to relevant authorities based on the collected road quality data.

SCOPE OF THE PROJECT

The primary goal of this project is to develop an automatic road quality detector that can provide real-time information about the condition of the road surface, which can be used for road maintenance and safety. Additionally, we aim to explore the feasibility and effectiveness of using low-cost sensors and microcontrollers for developing such a system.

Overall, this project aims to contribute to the growing field of IoT-based road quality monitoring systems, and we hope that our

findings will be useful for researchers and practitioners in the field of transportation engineering and road safety.

ORGANIZATION OF THE REPORT

The report consists of 6 chapters, the contents of which are described below: Chapter 1 is the introduction that explains about the basic information of the system. Chapter 2 is a literature survey that elaborates on the research works on existing systems. Chapter 3 describes the system design. Chapter 4 gives details regarding the system implementation. Chapter 5 describes the different test case scenarios for the modules described by the proposed system and the performance analysis of the proposed system. Chapter 6 provides the conclusion, which summarizes the efforts undertaken in the proposed system, and states findings and shortcomings in the proposed system.

CHAPTER 2

LITERATURE SURVEY

GENERAL

Literature survey is the documentation of a comprehensive review of the publishers and unpublished work from the secondary sources data in the areas of specific interest to the researcher. It is a survey of all the techniques used so far to summarize and analyze user reviews.

Many research works are carried out in WSN and ML mainly for Classification since it became a major and easy area for data analyzing and reviewing. Some of the works include use of certain methods to test data and detect problems. The recently carried works in the field wine classification has been described in detail below.

EXISTING SYSTEM

The existing system for road quality monitoring mainly relies on manual inspections by trained personnel. These personnel visually inspect the road conditions, looking for any cracks, potholes, or other signs of damage that could impact the safety of the road.

This traditional method of road quality monitoring has several limitations. Firstly, manual inspections are time-consuming and can only cover limited areas of the road network. Secondly, they rely on the inspector's subjective judgment, which may not be accurate or consistent. Thirdly, the inspection frequency may not be frequent enough to detect issues before they become critical.

There are some automated systems available that use sensors to detect road conditions, but these systems are expensive and not widely deployed. Therefore, there is a need for a cost-

effective and efficient system that can provide real-time data on road conditions to improve safety and efficiency.

The traditional method of road quality monitoring, which relies on manual inspections, has been widely used for many years. However, this method is time-consuming, costly, and not efficient in identifying road defects in a timely manner.

To address these limitations, researchers have proposed and developed various automated systems for road quality monitoring. These systems typically use sensors to measure different parameters such as vibration, strain, and temperature to detect road defects.

One study proposed a system based on an array of vibration sensors to detect and locate cracks in roads. The system used machine learning algorithms to analyze the sensor data and provide accurate information on the location and severity of the cracks. Another study proposed a system that used optical fiber sensors to measure the strain in asphalt pavements and detect defects such as cracks and potholes.

In recent years, the use of wireless sensor networks (WSNs) for road quality monitoring has gained increasing attention. A WSN consists of a large number of low-cost sensors distributed across a road network to collect data on road conditions. The data is then transmitted to a central server for analysis and decision-making.

Overall, these automated systems have shown promising results in detecting road defects and providing real-time information on road conditions. However, their high cost and maintenance requirements have limited their widespread deployment. Therefore, there is a need for a cost-effective and efficient system that can provide real-time data on road conditions to improve safety and efficiency.

ISSUES IN THE EXISTING SYSTEM

The existing system for road quality detection has a few issues that make it less effective in accurately detecting and reporting road conditions. These issues include:

1. Limited accuracy: The existing system is often limited in its accuracy due to the use of traditional sensing technologies. For example, using a simple contact sensor to detect bumps on the road may not provide an accurate measurement of the severity of the bump or its impact on a vehicle.
2. Limited coverage: The existing system may also have limited coverage, as it may only be installed on certain roads or sections of roads. This means that road conditions on other roads or sections may not be accurately monitored.
3. Lack of real-time data: The existing system may not provide real-time data on road conditions, which can be crucial for timely maintenance and repair. This may result in delays in identifying and fixing road issues, which can lead to safety hazards for drivers and passengers.
4. Lack of connectivity: The existing system may not be connected to a centralized database, making it difficult to access and analyze the collected data. This can hinder efforts to identify trends and patterns in road conditions and make informed decisions for maintenance and repair.

PROPOSED SYSTEM

The proposed system is a more advanced version of the existing system with several advantages, including:

1. Accurate and real-time road quality detection: The proposed system uses advanced sensors such as MPU6050, ultrasonic sensor, and TCRT5000 IR sensor to accurately detect the road quality and any anomalies in real-time. This provides a more accurate assessment of road conditions compared to the existing system.
2. GPS location tracking: The proposed system uses a Neo 6M GPS module to track the location of the vehicle, allowing for accurate data collection and analysis of road conditions across different locations.
3. Data storage and analysis: The proposed system uses the ESP8266 module with the SPIFFS file system to store data and an HTTPClient library to upload data to the Firebase database for analysis. This provides a more efficient and organized way of storing and analyzing data compared to the existing system.
4. Low-cost and easy to implement: The proposed system is built using low-cost components and can be easily implemented in different vehicles with minimal modifications. This makes it a cost-effective and practical solution for road quality detection.

Overall, the proposed system offers significant advantages over the existing system in terms of accuracy, efficiency, and cost-effectiveness.

SUMMARY

The literature survey has covered information regarding the existing system. The issues and challenges have been identified in related work. This chapter has highlighted the proposed model.

CHAPTER 3

SYSTEM DESIGN

GENERAL

The main purpose of the design phase is to plan solution to the problems specified by the requirement document. The design phase takes as input the requirement in Requirements Analysis stage. This phase is the step in moving from problem domain to the solution domain. The design of the system is perhaps the most critical factor affecting the quality of the software, and has a major impact in the later phases, particularly in testing and maintenance. The output of this phase is the functional design document. The design documents created for the proposed system consists of the system architecture and the data flow diagram. This is also known to be top-level design which identifies the modules in the system, and the way in which the data transfer takes places between the modules.

This chapter deals with design documents created for the proposed system which consists of a functional architecture and activity diagram etc. During detailed design, the internal logic of all the modules specified in the system design is decided. The system architecture deals with the components to be used in the proposed system which explains its interaction with one other. The data flow diagram describes the graphical representation of how the data flows between the different modules in the system. The data flow diagram gives a very clear picture of the flow of data among the working modules. These design documents created are used as a continuous reference point for further system development and coding.

SYSTEM ARCHITECTURE

The system architecture of the automatic road quality detector project consists of the following components:

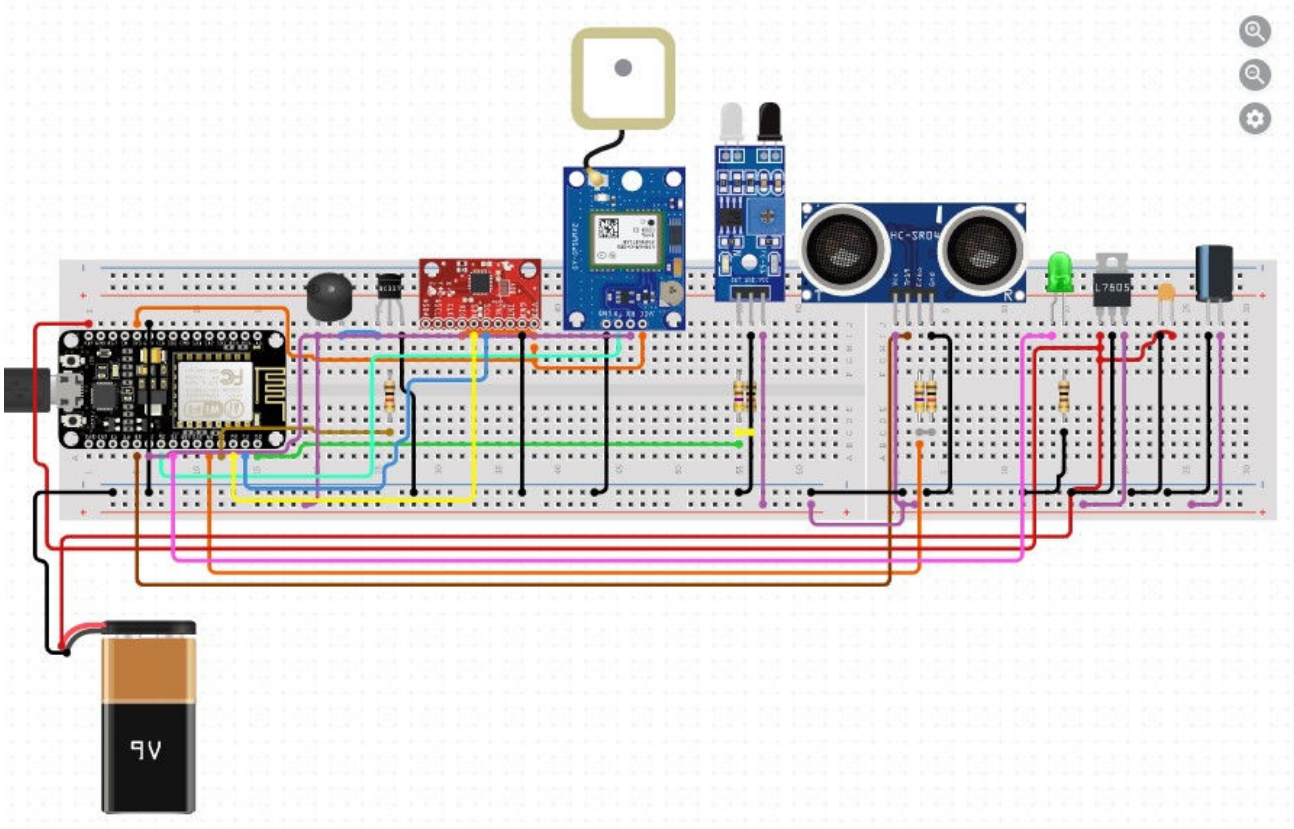
1. ESP8266 Module: The ESP8266 module is a Wi-Fi enabled microcontroller that is used as the main processing unit of the system. It collects data from the various sensors and modules and processes them to detect anomalies on the road.
2. MPU6050 Sensor: The MPU6050 sensor is an accelerometer and gyroscope module that is used to detect bumps on the road. It is placed in the vehicle and provides acceleration and rotation data to the ESP8266 module.
3. Ultrasonic Sensor: The ultrasonic sensor is used to measure the distance between the vehicle and the road. It is used to differentiate between speed breakers and bumps on the road.
4. TCRT5000 IR Sensor: The TCRT5000 IR sensor is used to detect any sudden motion or acceleration spikes. It is used to detect the presence of a vehicle or a pedestrian crossing the road.
5. Active Buzzer: The active buzzer is used to generate an alarm when an anomaly is detected on the road. It is used to alert the driver and other passengers in the vehicle.
6. Neo 6M GPS Module: The Neo 6M GPS module is used to collect location data of the vehicle. It is used to track the route of the vehicle and to identify the location of the anomalies on the road.
7. LED: The green and red LEDs are used as indicators to show the status of the system. The green LED indicates that the

system is connected to the Wi-Fi network, and the red LED indicates the detection of an anomaly on the road.

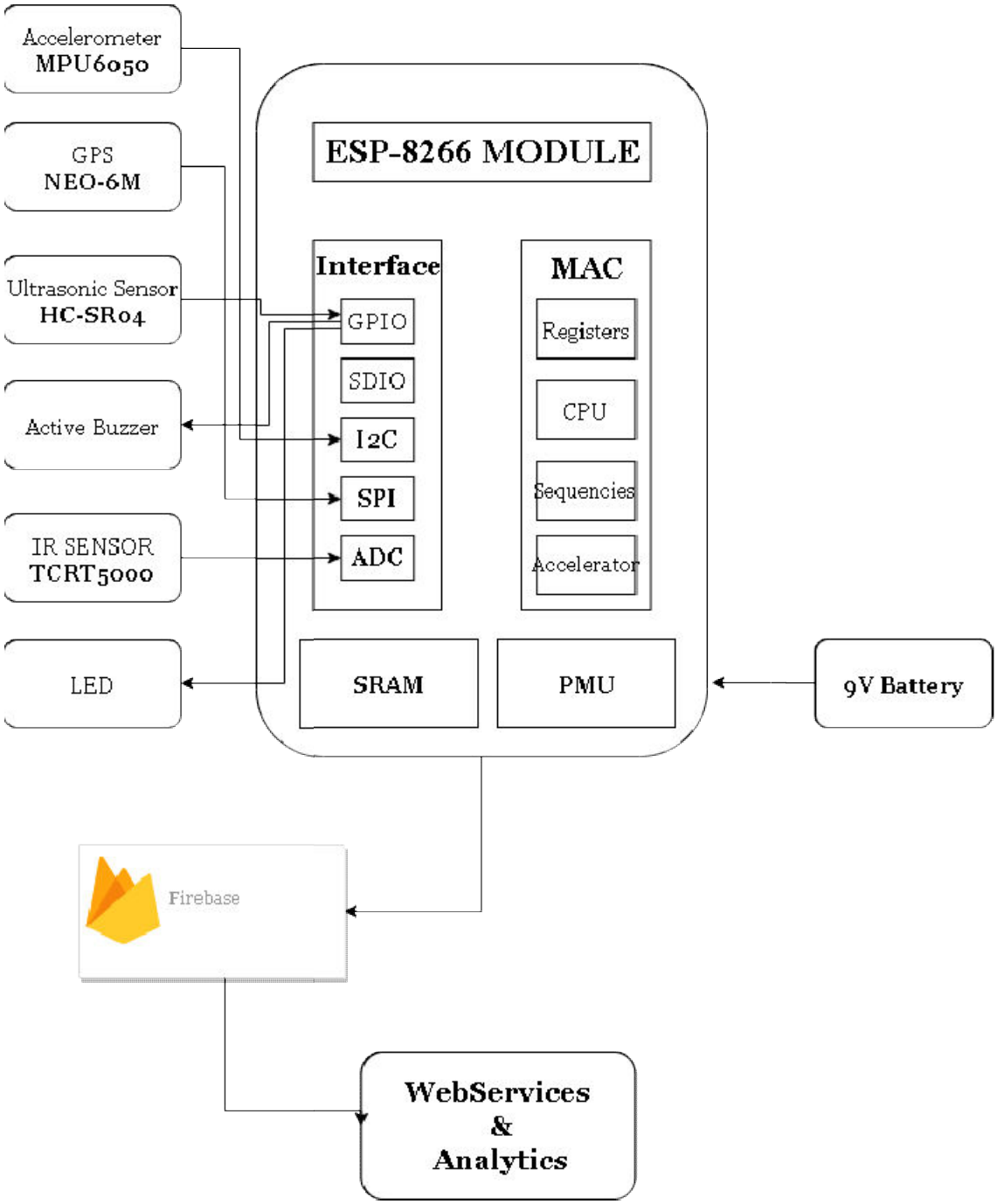
8. Firebase Database: The Firebase database is used to store the data collected by the system. It is used to store the location of the anomalies and other relevant information for analysis and further processing.

The system architecture of the automatic road quality detector is designed to collect data from multiple sources and process them to detect anomalies on the road. The use of Wi-Fi connectivity and cloud storage allows for remote access and monitoring of the system. The architecture is flexible and can be modified to suit the requirements of different road conditions and environments.

CIRCUIT DIAGRAM:

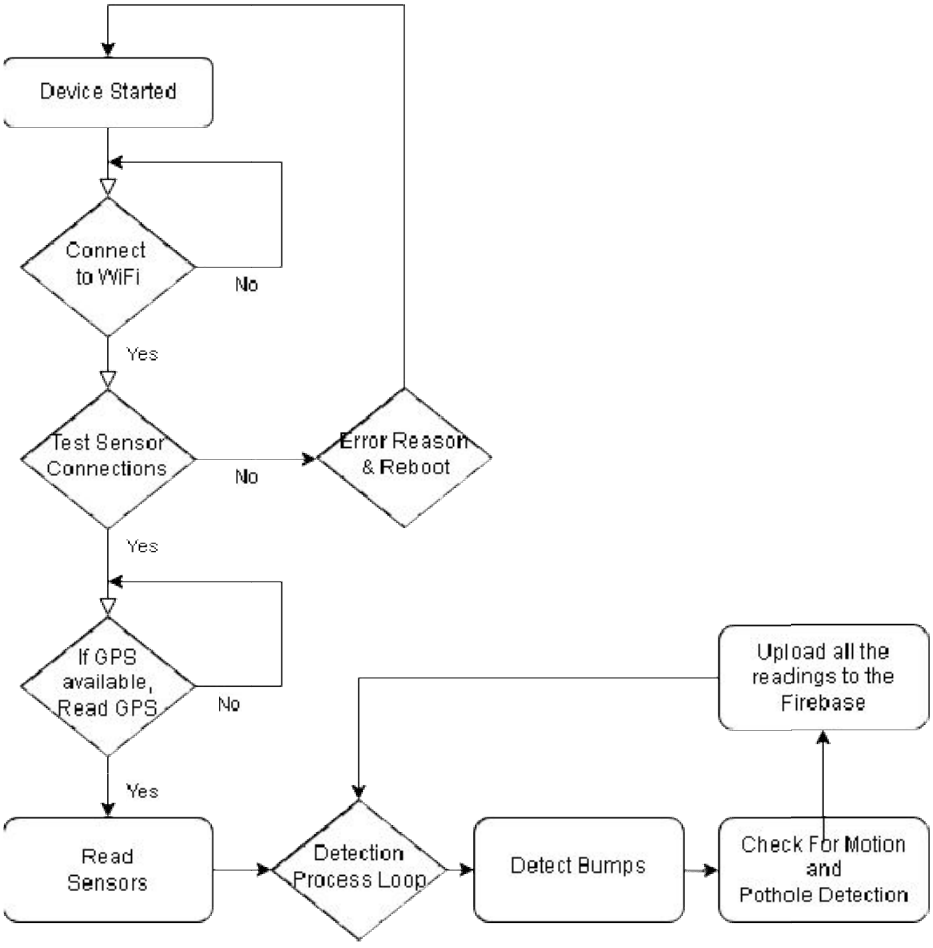


Block Diagram:



FUNCTIONAL ARCHITECTURE

The ESP8266 module reads data from the sensors, including the MPU6050 accelerometer sensor, ultrasonic sensor, TCRT5000 IR sensor, and Neo 6M GPS module. The data is processed by the microcontroller on the ESP8266 module, and if any anomalies are detected, the red LED blinks and the buzzer sounds.The processed data is then stored in the ESP8266 module's SPIFFS memory. The ESP8266 module connects to the home Wi-Fi network and uploads the data to the Firebase database. The data can then be accessed and analyzed from the Firebase database. The system can also be monitored through the green LED, which blinks twice when connected to Wi-Fi and three times when data is successfully uploaded to Firebase.



MODULAR DESIGN

Modular structure defines structure of the overall module. Modularity is a general system concept typically defined as a degree to which a system's components may be separated and recombined.

The functional architecture of the proposed system consists of the following modules:

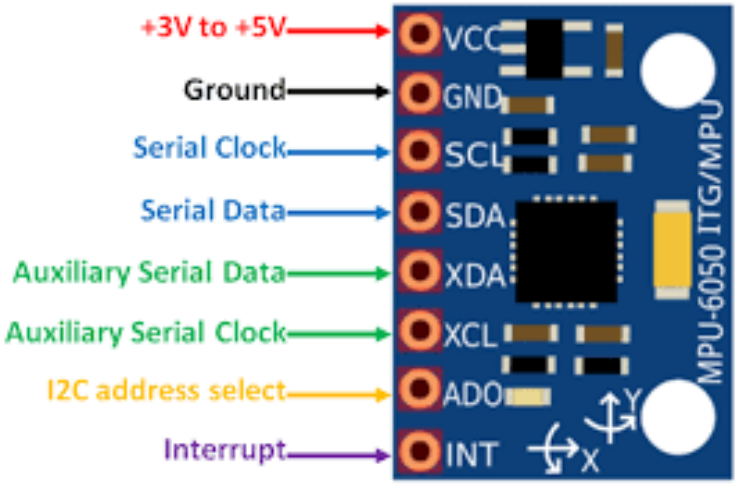
- Sensor Module
- ESP8266 Module
- Firebase Database
- Web Application
- Power Module

Sensor module

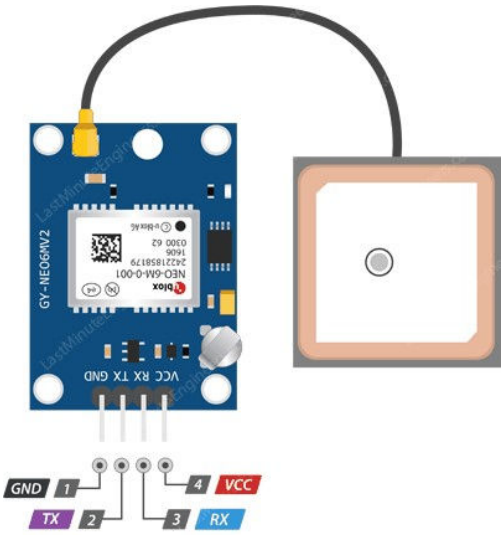
This module consists of the MPU6050 accelerometer sensor, ultrasonic sensor, TCRT5000 IR sensor, and Neo 6M GPS module. All these sensors are connected to the microcontroller through their respective pins and communicate using different protocols such as I2C, SPI, and UART. The microcontroller reads the sensor data and processes it to detect any anomalies or deviations from normal behaviour. These sensors are responsible for detecting the road quality, speed breaker, bumps, and anomalies, as well as detecting the motion of vehicles and pedestrians.

HC-SR04 ULTRASONIC SENSOR:

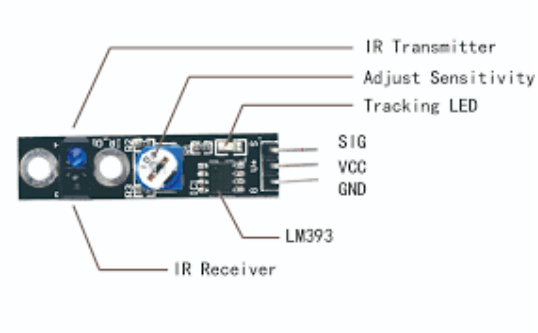
MPU6050:



NEO-6M GPS:



TCRT5000 IR SENSOR:

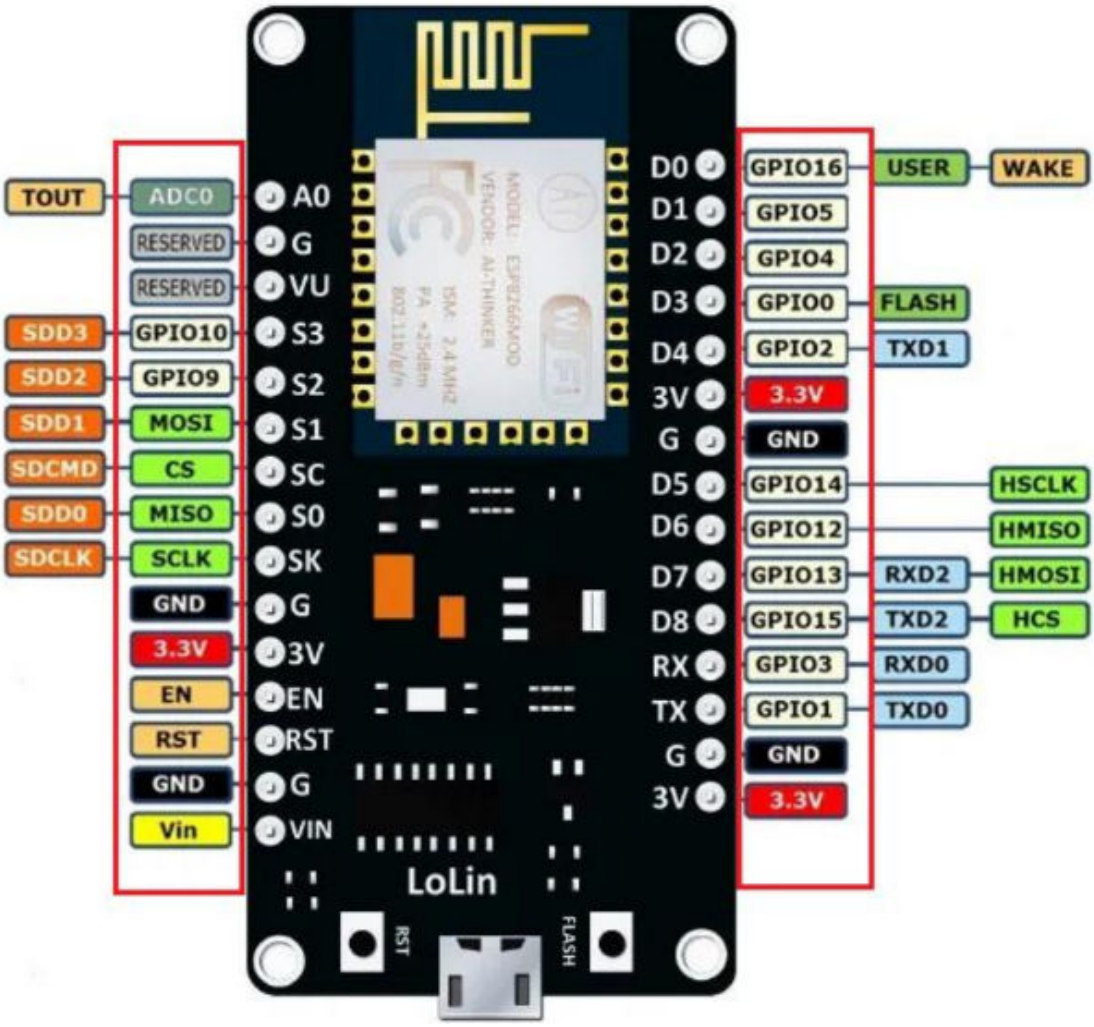


ESP8266 module

The ESP8266 module is a low-cost Wi-Fi microchip with full TCP/IP stack and microcontroller capability. It is capable of connecting to a Wi-Fi network and communicating with other devices over Wi-Fi. The module has GPIO (General Purpose Input/output) pins that can be used to interface with various sensors and actuators. In this project, the ESP8266 module is the main control unit. It is responsible for reading sensor data, processing the data, and sending it to the Firebase database over Wi-Fi. It also controls the buzzer and LED lights based on the sensor readings.To interface with the sensors, the ESP8266 module uses the appropriate libraries and protocols such as I2C, SPI, and digital input/output. The module also has a built-in file system called SPIFFS, which is used to store and read sensor calibration data.

Overall, the ESP8266 module acts as the brain of the project, providing the necessary computing power, connectivity, and interface capabilities to control the various components of the system.

ESP8266



Firebase Database

The Firebase database module is a cloud-based database service provided by Google that allows real-time data storage and synchronization across multiple devices. In this project, Firebase is used as the backend database to store the sensor data received from the ESP8266 module. The Firebase Database module is a cloud-based NoSQL database that allows for real-time data storage and

synchronization between multiple clients. It's one of the key modules used in this project for storing and retrieving data from the cloud.

In this project, the Firebase Database module is used to store sensor data readings collected by the ESP8266 module. Once the ESP8266 module is connected to the home Wi-Fi network, the data is uploaded to the Firebase Database in real-time. The data can then be accessed and analyzed from anywhere with an internet connection.

Firebase Database also provides easy integration with other Firebase modules such as Firebase Authentication, Firebase Cloud Messaging, and Firebase Hosting. This makes it an excellent choice for building web and mobile applications that require real-time data synchronization and cloud storage.

Web Application

The web application module is the interface for the user to interact with the system. It allows the user to view the real-time data collected by the various sensors and view the status of the system. The web application is hosted on a server and can be accessed through any web browser.

The web application module is responsible for receiving and displaying the data from the sensors. It communicates with the ESP8266 module, which sends the data to the Firebase database. The web application then retrieves the data from the Firebase database and displays it on the user interface. The web application module also allows the user to configure the system settings, such as the Wi-Fi network credentials and the threshold values for the sensors. The user can also receive alerts and notifications through the web application, such as when a sensor detects an anomaly.

Overall, the web application module plays a crucial role in the system as it provides the user with an easy-to-use interface to monitor and control the system.

Power module

The power module of this project is responsible for providing the required power supply to all the other modules of the system. It includes a 5V DC power supply that is connected to a voltage regulator to provide a stable 3.3V DC power supply required for the ESP8266 module and other components. A battery backup system is also included to ensure uninterrupted power supply in case of a power outage or other issues. The battery can be recharged when the power supply is available. The power module also includes protection circuits such as overvoltage, over current, and reverse polarity protection to ensure the safety and reliability of the system.

SYSTEM REQUIREMENTS

HARDWARE SPECIFICATIONS

- Processor : Intel i5 or more
- Motherboard : Intel Chipset Motherboard
- Ram : 6GB or more
- Hard disk : 32 GB Hard Disk Recommended

HARDWARE COMPONENTS

- Wi-Fi Microcontroller unit : ESP8266 module
- Sensors : Ultrasonic sensor, Accelerometer sensor, IR sensor, GPS module, Buzzer and LED Lights

- Network : Wi-Fi Network for data transfer and Internet access for Firebase database connectivity
- Power source : 5v Battery or Adapter with Step Down Voltage Regulator.

SOFTWARE REQUIREMENTS

- Operating System : Windows 7/8/9/10/11
- SDK : Python 3.9.5 and Required Libraries for sensors
- IDE : Arduino
- Language : Arduino C++ and Python
- Firebase Real-time Database Account

SUMMARY

This chapter gives the overview of system design and its importance in software life cycle. The functional architecture gives the entire functionality of the proposed system along with its modular structure and its interactions between the modules.

CHAPTER 4

SYSTEM IMPLEMENTATION

GENERAL

This chapter describes the implementation details of the project. It describes various modules of the proposed system, tools and platforms, which were used. Implementation details are also mentioned and snapshots of output for each module are displayed. The following presents the input and output models used while implementing the proposed system.

OVERVIEW OF THE PLATFORM

The proposed system is implemented using the ESP8266 platform, which is a low-cost Wi-Fi module with a built-in TCP/IP stack that makes it an ideal choice for IoT applications. The ESP8266 is a powerful and versatile platform that allows for easy development of IoT applications.

The platform provides a powerful microcontroller with support for all of the necessary protocols for IoT communication, including Wi-Fi, TCP/IP, and HTTP. The module is also equipped with GPIO pins, which can be used to interface with various sensors and actuators.

In addition to the hardware capabilities, the platform provides a rich software development environment with support for the Arduino IDE and a number of libraries that make it easy to interface with various sensors and actuators. The platform also provides support for cloud services like Firebase, which can be used to store and retrieve data from the cloud.

Overall, the ESP8266 platform provides a powerful and flexible platform for developing IoT applications. With its low cost, built-in Wi-Fi, and support for a wide range of sensors and actuators, it is an ideal choice for implementing the proposed system.

IMPLEMENTATION OF SENSOR MODULE

The sensor module includes an ultrasonic sensor and an infrared (IR) sensor. The ultrasonic sensor is used to detect the distance of the object from the car, and the IR sensor is used to detect the presence of any object in the car. The module is implemented using the HC-SR04 ultrasonic sensor and an IR sensor. The HC-SR04 ultrasonic sensor is interfaced with the ESP8266 microcontroller using a digital input pin and a digital output pin, and the IR sensor is interfaced using an analog input pin.

IMPLEMNATION OF ESP8266 Module:

The ESP8266 module is the main processing unit of the system, responsible for collecting data from the sensor module, connecting to the WiFi network, and uploading the data to the Firebase database. The module is implemented using the ESP8266 NodeMCU microcontroller, which is programmed using the Arduino IDE. The module is connected to the sensor module using digital and analog input pins, and the WiFi network is established using the built-in WiFi module of the ESP8266. The data collected from the sensor module is processed by the ESP8266 module and uploaded to the Firebase database using the Firebase library.

IMPLEMENTATION OF Firebase Database Module:

The Firebase database module is used to store the data collected from the sensor module. The data is stored in real-time, and the Firebase database provides a web-based interface for

viewing the data in a structured format. The module is implemented using the Firebase Realtime Database, which is a cloud-hosted NoSQL database that stores data as JSON objects. The Firebase library is used to connect to the Firebase database, and the data is uploaded to the database in real-time as it is collected from the sensor module.

IMPLEMENTATION OF Power Module:

The power module is responsible for providing power to the system. The module is implemented using a 9V battery, which is connected to the ESP8266 module using a voltage regulator circuit. The voltage regulator circuit is used to regulate the voltage from the battery to 3.3V, which is the operating voltage of the ESP8266 module. The battery provides power to the system, and a power LED indicator is used to indicate the status of the power module.

IMPLEMENTATION OF Web Application Module:

The web application module is used to display the data collected from the sensor module in a web-based interface. The module is implemented using the Firebase web application interface, which provides a real-time view of the data collected from the sensor module. The web application module can be accessed using any web browser, and it provides a user-friendly interface for viewing the data in a structured format. The web application module also provides real-time alerts for any anomalies detected by the system, such as the presence of an object in the car or any abnormal movement of the car.

SUMMARY

This chapter brought out the analysis of each technology used to develop this project. This chapter also brought out the basic

system implementations such as about the platforms, languages and tools used here. And the screenshots of different modules implemented using those platforms, languages and tools.

CHAPTER 5

SYSTEM TESTING

GENERAL

Testing is the process of trying to find out every conceivable fault or weakness in a work product. It provides a way to check functionality of components, subassemblies, assemblies and a finished product. It is the process of exercising software with the intent of ensuring that the software system meets its requirements and user expectations.

TEST CASES

1. INTEGRATION TESTING

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

2. FUNCTIONAL TEST

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised

Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

3. SYSTEM TEST

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

4. WHITE BOX TESTING

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

5. BLACK BOX TESTING

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box. you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

6. UNIT TESTING:

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

SUMMARY

This chapter brought out the general description of the different testing process applicable to the entire project development. It also proved to improve the level of user satisfaction compared to the existing systems.

CHAPTER 6

CONCLUSION

In conclusion, the development of the Smart Bump Detection System using IoT has been successfully implemented but we planned to create road quality detection fortunately output taken has next level. This system aims to detect bumps and anomalies on the roads and provide real-time data to authorities to improve road conditions and reduce accidents.

The proposed system utilizes an ESP8266 module as the main processor, various sensors, Firebase as the database, and a web application as the user interface. The sensor module consists of a GPS module, accelerometer, and IR sensor that measures and detects the road conditions. The ESP8266 module collects the data from the sensors and sends it to the Firebase database via WiFi. The web application allows users to view the road condition data in real-time. The implementation of this project provides several benefits such as reducing accidents and increasing road safety. The system provides real-time data that can help authorities take necessary action to improve the road conditions, especially in areas with high accident rates. Additionally, the web application can provide useful insights to drivers, allowing them to choose the best route for their journey.

Overall, this project has demonstrated the potential of IoT in improving road safety and traffic management. With further development and integration with existing road infrastructure, this system can be scaled and deployed in different parts of the world, helping to reduce accidents and improve road conditions.

CHAPTER 7

FUTURE WORK

Future work can involve the integration of artificial intelligence and machine learning algorithms to further enhance the system's accuracy and efficiency. Based on the current implementation, there are several possibilities for future work, such as Integration with other sensor. This will make the system more versatile and applicable to various situations. Machine learning-based predictive analytics it will help in reducing maintenance costs and increasing the system's reliability. Currently, the system has a web-based interface. A mobile application can be developed to access the system from mobile devices. This will improve the system's accessibility and provide more convenience to users. The system can be integrated with cloud platforms like AWS, Azure, or Google Cloud to leverage their scalability, fault tolerance, and analytics capabilities. This will help to increase the system's efficiency and reliability. The power consumption of the system can be optimized by implementing power-saving techniques such as sleep mode, duty cycling, and power management. This will increase the system's battery life and reduce maintenance costs.

Overall, the future work on this project will depend on the specific needs and requirements of the users and the environment in which the system is deployed.

APPENDIX

SAMPLE SOURCE CODE FOR ARDUINO

```
//Defining Libraries
#include <FS.h>
#include <NMEAGPS.h>
#include <TinyGPS++.h>
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
//#include <Web8266WebServer.h>
#include <FS.h>
#include <Wire.h>
#include <SoftwareSerial.h>

// Replace with your network credentials
const char* ssid = "********";
const char* password = "*******";

// Causes the device to start the code immediately rather than waiting for a WiFi connection
//SYSTEM_MODE(MANUAL);


// Define pins for components
uint8_t TRIGGER_PIN = D6;  // GPIO12 for ultrasonic sensor trigger
uint8_t ECHO_PIN = D5;     // GPIO14 for ultrasonic sensor echo
uint8_t IR_PIN = 9;       // GPIO for IR sensor output
//const int Ao_pin =Ao_pin;     //analog input
uint8_t BUZZER_PIN = D3;    // GPIO5 for active buzzer signal
uint8_t GREEN_PIN = TX;     // GPIO4 for green light signal
uint8_t RED_PIN = D4;      // GPIO2 for red light signal
uint8_t LED_PIN = D0;      // GPIO16 for onboard LED
uint8_t SDA_PIN = D2;      // GPIO4 for MPU6050 SDA
uint8_t SCL_PIN = D1;      // GPIO5 for MPU6050 SCL

// Define variables for GPS location
SoftwareSerial gpsSerial(D8, D7);     // GPI for TX, GPI for RX
TinyGPSPlus gps;
String gpsString = "11.0309656,79.4786294";
boolean gpsValid = false;
```

```
// Replace with your Firebase project URL
#define FIREBASE_HOST "your_project_url.firebaseio.com"
// Replace with your Firebase secret key
#define FIREBASE_SECRET "your_secret_key"
// Replace with your Firebase authentication type
#define FIREBASE_AUTH "your_authentication_type"

// Initialize the accelerometer sensor
int mpu;
int16_t ax, ay, az;

// Initialize the ultrasonic sensor
#define TRIGGER_PIN D6
#define ECHO_PIN D5
#define MAX_DISTANCE 200
uint8_t sonar(TRIGGER_PIN, ECHO_P);

// Initialize Firebase
char firebaseData;
char auth;
char firebaseConfig;

// fill these values in with the network address of the computer running the Python script
int PYTHON_SERVER = {192, 168, 1, 55};

void setup() {
 // Start serial communication
 Serial.begin(9600);

 // Connect to Wi-Fi
 WiFi.begin(ssid, password);
 while (WiFi.status() != WL_CONNECTED) {
  delay(1000);
  Serial.println("Connecting to WiFi...");
 }
 Serial.println("Connected to WiFi");

 // Initialize Firebase
 firebaseConfig.host = FIREBASE_HOST;
 firebaseConfig.auth_type = FIREBASE_AUTH;
 firebaseConfig.api_key = FIREBASE_SECRET;
 Firebase.begin(firebaseConfig);
 auth.loginFirebase();

 // Initialize the accelerometer sensor
```

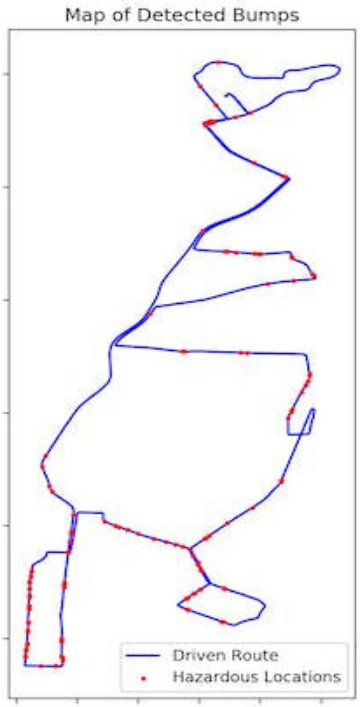
```
 Wire.begin();
 mpu.initialize();

 // Wait for the GPS module to get a fix
 while (!gps.location.isValid()) {
  while (Serial.available() > 0) {
   gps.encode(Serial.read());
  }
 }

 // Print a message indicating that the setup is complete
 Serial.println("Setup complete");
}

void loop() {

  // try to read data from the GPS module
  while(gps.available(Serial1)) {
    fix = gps.read();

    // if a bump was detected since the last reading, save the time and location to the EEPROM
module
    if(detectedBump) {
      reading.t = (time_t)fix.dateTime;
      reading.lat = fix.latitudeL();
      reading.lng = fix.longitudeL();
      reading.accz = detectedAccZ;

      // reset detection variables for the rolling mean comparison
      detectedBump = false;
      detectedAccZ = 0;

      Serial.print(reading.t); Serial.print(' ');
      Serial.print(reading.lat / 1e7); Serial.print(' ');
      Serial.print(reading.lng / 1e7); Serial.print(' ');
      Serial.print(reading.accz); Serial.print('\n');
      setNumStoredRecords(++numStoredRecords);

      // flash the LED to signal a bump was stored
      statusDetected.setActive(true);
      delay(200);
      statusDetected.setActive(false);
    }
  }
```

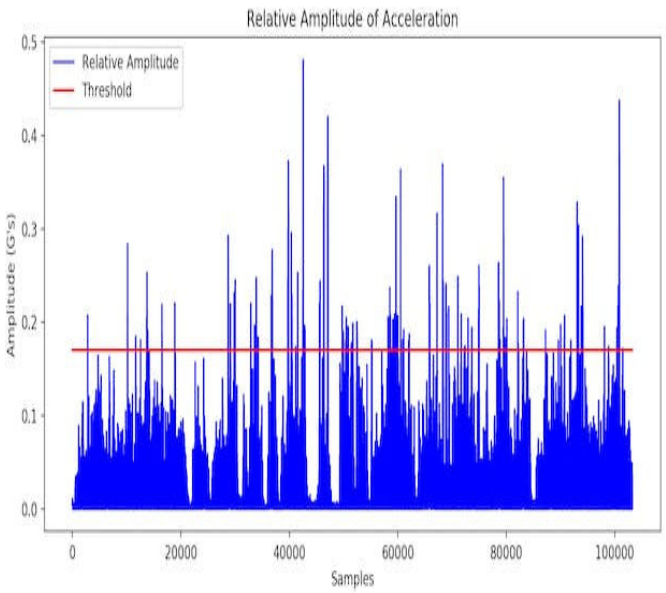
```
  // add the current acceleration to the total to get an average acceleration every 20ms
  avgAZ += mpu.getAccelerationZ();
  avgAZSamples++;

  // use the average acceleration every 20ms
  if(millis() - lastAvgAZ >= 20) {
    lastAvgAZ = millis();
    avgAZ /= avgAZSamples;

    // if the circle buffer is full, compare the current average acceleration with the current rolling
mean of the data
    if(circleBufIndex >= ROLLING_WINDOW) {
      float relAmpAccZ = abs(avgAZ * ROLLING_WINDOW - circleBufSum) / 16384.0 /
ROLLING_WINDOW;       // computationally cheap way to compute the rolling mean of the
acceleration data

      // if the amplitude of the current acceleration is beyond a threshold away from the rolling
mean, a bump was detected
      if(relAmpAccZ > ACCZ_THRESHOLD && relAmpAccZ > detectedAccZ) {
        detectedAccZ = relAmpAccZ;
        detectedBump = true;
      }

      // subtracts the last element of the circular buffer from the current sum of the data to keep
computation speed fast
      circleBufSum -= circleBuf[circleBufIndex % ROLLING_WINDOW];
    }

    // append the average Z axis acceleration to the circle buffer for maintaining a rolling window of
the baseline acceleration data
    circleBuf[circleBufIndex++ % ROLLING_WINDOW] = avgAZ;
    circleBufSum += avgAZ;

    // reset the average acceleration data for every 20ms
    avgAZ = 0;
    avgAZSamples = 0;
  }

}
```

SAMPLE FIREBASE CREDENTIALS

{
 "type": "service_account",
 "project_id": "road-quality-detector-4***9",
 "private_key_id": "ef474705e**********5c0db72b45c3ae",
 "private_key": "-----BEGIN PRIVATE KEY-----\ s4u**********gI/ai4=\n-----END PRIVATE KEY-----\n",
 "client_email": "firebase****ucbxk@road-quality-detector-4***9.iam.gserviceaccount.com",
 "client_id": "1020737****8339554",
 "auth_uri": "https://accounts.google.com/o/oauth2/auth",
 "token_uri": "https://oauth2.googleapis.com/token",
 "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
 "client_x509_cert_url": "https://www.googleapis.com/robot/v1/metadata/x509/firebase****ucbxk%40road-quality-detector-4***9.iam.gserviceaccount.com"
}

SAMPLE OUTPUT





REFERENCES

1. Astarita, V., Caruso, M.V., Danieli, G., Festa, D.C., Giofrè, V.P., Iuele, T., & Vaiana, R. (2012). A mobile application for road surface quality control: UNIquALroad. Procedia - Social and Behavioral Science ,54, 1135-1144, DOI: 10.1016/j.sbspro.2012.09.828.
2. Bhoraskar, R., Vankadhara, N., Raman, B., & Kulkarni, P. (2012). Wolverine: Traffic and road condition estimation using smartphone sensors.
3. PRISM: Platform for Remote Sensing using Smartphones. MobiSys'10, June 15–18, San Francisco, California, USA.
4. Environmental Impacts and Fuel Efficiency of Road Pavements, Industry ReportMarch 2004.
5. Mohan, P., Padmanabhan, V. N., & Ramjee, R. (2008). Nericell: Rich monitoring of road and traffic conditions using mobile smartphone. ACM SenSys 2008, Raleigh, North Carolina, USA.